

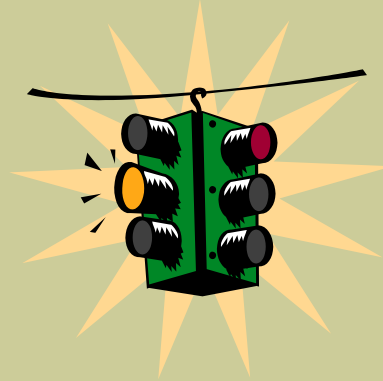
Programación Concurrente

Clase 3



Facultad de Informática
UNLP

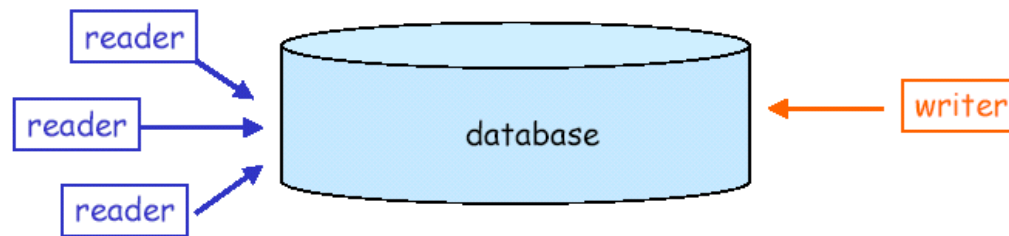
Semáforos (continuación)



Problemas básicos y técnicas

Lectores y escritores

- **Problema:** dos clases de procesos (*lectores* y *escritores*) comparten una Base de Datos. El acceso de los *escritores* debe ser exclusivo para evitar interferencia entre transacciones. Los *lectores* pueden ejecutar concurrentemente entre ellos si no hay escritores actualizando.



- Procesos asimétricos y, según el scheduler, con diferente prioridad.
- Es un problema de ***exclusión mutua selectiva***: procesos que compiten por el acceso a conjuntos superpuestos de variables compartidas.
- Diferentes soluciones:
 - Como problema de exclusión mutua.
 - Como problema de sincronización por condición.

Problemas básicos y técnicas

Lectores y escritores: *como problema de exclusión mutua*

- Los escritores necesitan acceso mutuamente exclusivo.
- Los lectores (como grupo) necesitan acceso exclusivo con respecto a cualquier escritor.

```
sem rw = 1;  
process Lector [i = 1 to M]  
{ while(true)  
  { ...  
    P(rw);  
    lee la BD;  
    V(rw);  
  }  
}  
process Escritor [j = 1 to N]  
{ while(true)  
  { ...  
    P(rw);  
    escribe la BD;  
    V(rw);  
  }  
}
```

No hay concurrencia entre lectores

- Los lectores (como grupo) necesitan bloquear a los escritores, pero sólo el primero necesita tomar el *lock* ejecutando $P(rw)$.
- Análogamente, sólo el último lector debe hacer $V(rw)$.

Problemas básicos y técnicas

Lectores y escritores: *como problema de exclusión mutua*

```
int nr = 0;      # número de lectores activos
sem rw = 1;      # bloquea el acceso a la BD
```

```
process Escritor [j = 1 to N]
{ while(true)
  { ...
    P(rw);
    escribe la BD;
    V(rw);
  }
}
```

```
process Lector [i = 1 to M]
{ while(true)
  { ...
    < nr = nr + 1; if (nr == 1) P(rw); >
    lee la BD;
    < nr = nr - 1; if (nr == 0) V(rw); >
  }
}
```

Problemas básicos y técnicas

Lectores y escritores: *como problema de exclusión mutua*

```
int nr = 0;           # número de lectores activos
sem rw = 1;           # bloquea el acceso a la BD
sem mutexR = 1;       # bloquea el acceso de los lectores a nr
```

```
process Escritor [j = 1 to N]
{ while(true)
  { ...
    P(rw);
    escribe la BD;
    V(rw);
  }
}
```

```
process Lector [i = 1 to M]
{ while(true)
  { ...
    P(mutexR);
    nr = nr + 1;
    if (nr == 1) P(rw);
    V(mutexR);
    lee la BD;
    P(mutexR);
    nr = nr - 1;
    if (nr == 0) V(rw);
    V(mutexR);
  }
}
```

Problemas básicos y técnicas

Lectores y escritores: *sincronización por condición*

- Solución anterior $\Rightarrow$ preferencia a los lectores $\Rightarrow$ no es *fair*.
- Otro enfoque $\Rightarrow$ pueden contarse (por medio de *nr* y *nw*) los procesos de cada clase accediendo a la BD, y luego restringir el valor de los contadores.

```
int nr = 0, nw = 0;

process Lector [i = 1 to M]
{ while(true)
  { ...
    < await (nw == 0) nr = nr + 1; >
    lee la BD;
    < nr = nr - 1; >
  }
}

process Escritor [j = 1 to N]
{ while(true)
  { ...
    < await (nr==0 and nw==0) nw=nw+1; >
    escribe la BD;
    < nw = nw - 1; >
  }
}
```

Problemas básicos y técnicas

Lectores y escritores: *sincronización por condición*

- En algunos casos, await puede ser implementada directamente usando semáforos u otras operaciones primitivas, pero no siempre...En este caso las guardas de los await se superponen en: para los escritores necesita que tanto *nw* como *nr* sean 0, mientras para lectores sólo que *nw* sea 0. Ningún semáforo podría discriminar entre estas condiciones → *Passing the baton*.
- En lugar de mantener el “orden” por proceso se hará por tipo de proceso. Cada tipo de proceso espera una condición distinta, por lo que se necesitan SÓLO dos semáforos para dormir a los procesos dependiendo de que condición esperan (que tipo de proceso es). Los llamaremos “r” y “w”.
- Y para saber cuantos procesos agregaremos dormidor usaremos dos enteros. Los llamaremos “dr” y “dw”.
- La “SC” que debe proteger el *baton* (e) es donde se usen las variables compartidas: *nr*, *nw*, *dr*, *dw*

Problemas básicos y técnicas

Lectores y escritores: *Técnica Passing the Baton*

```
int nr = 0, nw = 0;
```

```
process Lector [i = 1 to M]
{ while(true)
```

```
  { ...
    < await (nw == 0) nr = nr + 1; >
    lee la BD;
    < nr = nr - 1; >
  }
}
```

```
process Escritor [j = 1 to N]
```

```
{ while(true)
  { ...
    < await (nr==0 and nw==0) nw=nw+1; >
    escribe la BD;
    < nw = nw - 1; >
  }
}
```



```
int nr = 0, nw = 0;
int dr = 0, dw = 0;
sem e = 1;
sem r = 0, w = 0;
```

```
process Lector [i = 1 to M]
```

```
{ while(true)
  { P(e);
    if (nw > 0) {dr = dr+1; V(e); P(r); }
    nr = nr + 1;
    if (dr > 0) {dr = dr - 1; V(r); }
    else V(e);
    lee la BD;
    P(e);
    nr = nr - 1;
    if (nr == 0 and dw > 0)
      {dw = dw - 1; V(w); }
    else V(e);
  }
}
```

Problemas básicos y técnicas

Lectores y escritores: *Técnica Passing the Baton*

```
int nr = 0, nw = 0;
```

```
process Lector [i = 1 to M]
```

```
{ while(true)
  { ...
    < await (nw == 0) nr = nr + 1; >
    lee la BD;
    < nr = nr - 1; >
  }
}
```

```
process Escritor [j = 1 to N]
```

```
{ while(true)
  { ...
    < await (nr==0 and nw==0) nw=nw+1; >
    escribe la BD;
    < nw = nw - 1; >
  }
}
```

```
int nr = 0, nw = 0;
```

```
int dr = 0, dw = 0;
```

```
sem e = 1;
```

```
sem r = 0, w = 0;
```

```
process Escritor [j = 1 to N]
```

```
{ while(true)
  { P(e);
    if (nr > 0 or nw > 0)
      {dw=dw+1; V(e); P(w);}
    nw = nw + 1;
    V(e);
    escribe la BD;
    P(e);
    nw = nw - 1;
    if (dr > 0) {dr = dr - 1; V(r); }
    elseif (dw > 0) {dw = dw - 1; V(w); }
    else V(e);
  }
}
```

Problemas básicos y técnicas

Lectores y escritores: *Técnica Passing the Baton*

```
int nr = 0, nw = 0, dr = 0, dw = 0;
```

```
sem e = 1, r = 0, w = 0;
```

```
process Lector [i = 1 to M]
{ while(true)
  { P(e);
    if (nw > 0) {dr = dr+1; V(e); P(r); }
    nr = nr + 1;
    if (dr > 0) {dr = dr - 1; V(r); }
    else V(e);
    lee la BD;
    P(e);
    nr = nr - 1;
    if (nr == 0 and dw > 0)
      {dw = dw - 1; V(w); }
    else V(e);
  }
}
```

```
process Escritor [j = 1 to N]
{ while(true)
  { P(e);
    if (nr > 0 or nw > 0)
      {dw=dw+1; V(e); P(w);}
    nw = nw + 1;
    V(e);
    escribe la BD;
    P(e);
    nw = nw - 1;
    if (dr > 0) {dr = dr - 1; V(r); }
    elseif (dw > 0) {dw = dw - 1; V(w); }
    else V(e);
  }
}
```

Da preferencia a los lectores $\Rightarrow$ ¿Cómo puede modificarse?